## Lab Work for Week 10: Deploying a Django Application

---

**Objective**

By the end of this lab, students will:

1. Prepare a Django project for deployment in a production environment.
2. Configure settings for security and performance.
3. Deploy the application to a cloud platform (e.g., Heroku).

---

**Step-by-Step Lab Tasks**

---

**1. Prepare the Django Project for Deployment**

1. **Task**: Set up static files for production.
    - Update settings.py:

```
import os


BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


# Static files
STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

    - Run the collectstatic command to collect all static files in a single directory:

```
python manage.py collectstatic
```

2. **Task**: Disable debug mode for production.
    - Update settings.py:

```
DEBUG = False
ALLOWED_HOSTS = ['*']
```

    - **Note**: Replace '*' with the domain or IP address of your server in a real deployment.
3. **Task**: Install gunicorn for serving the app in production.

*pip install gunicorn*

---

**2. Create a Requirements File**

1. **Task**: Generate a requirements.txt file to list all project dependencies.

*pip freeze > requirements.txt*

2. **Why This Matters**: This file ensures that the same dependencies are installed in the deployment environment.

---

**3. Create a Procfile for Heroku Deployment**

1. **Task**: Create a Procfile in the project's root directory.
    - Add the following content:

*web: gunicorn EduLearn.wsgi --log-file -*

2. **Why This Matters**: The Procfile tells Heroku how to start the application.

---

**4. Deploy to Heroku**

1. **Task**: Install the Heroku CLI and log in.
    - Install the CLI:
        - Heroku CLI Installation Guide
    - Log in:

*heroku login*

2. **Task**: Create a new Heroku app.

*heroku create your-app-name*

3. **Task**: Add a PostgreSQL database (optional for production-ready apps).

*heroku addons:create heroku-postgresql:hobby-dev*

4. **Task**: Push the project to Heroku.

- Initialize a Git repository if not already done:

*git init*

*git add .*

*git commit -m "Initial commit for deployment"*

- Set the Heroku remote:

*heroku git:remote -a your-app-name*

- Deploy the application:

*git push heroku main*

5. **Task**: Run database migrations on Heroku.

*heroku run python manage.py migrate*

6. **Task**: Create a superuser for the live site.

*heroku run python manage.py createsuperuser*

---

**5. Test the Deployed Application**

1. **Task**: Open the application in a browser.

*heroku open*

2. **Task**: Verify:
   - The homepage displays correctly.
   - The admin panel is accessible.
   - API endpoints work as expected.

---

**Bonus: Configure Environment Variables**

1. **Task**: Store sensitive settings like SECRET_KEY in environment variables.
   - Add the following to settings.py:

*import os*

*SECRET_KEY = os.getenv('SECRET_KEY', 'default-secret-key')*

- Set the variable on Heroku:

*heroku config:set SECRET_KEY=your-secret-key*

2. **Why This Matters**: Keeping sensitive information out of the codebase enhances security.

---

**Lab Deliverables**

1. A deployed application URL (e.g., https://your-app-name.herokuapp.com).
2. Code for:
   - Updated settings.py for production.
   - requirements.txt.
   - Procfile.
3. Screenshots of:
   - The live application homepage.
   - The admin panel on Heroku.
   - A successful API response from the live deployment.

---

**Additional Questions for Students**

1. What is the purpose of the collectstatic command in Django?
2. Why is it important to disable DEBUG in a production environment?
3. What are the benefits of using a platform like Heroku for deployment?